

Object Oriented Programming (CSC166)

Lab Sheet 1

1. Write a program that uses two namespaces to calculate salary for "Permanent" and "Contract" employees differently, and display the result neatly formatted using stream manipulators.

Permanent::calculateSalary → *basic + 20% HRA – 10% tax*

Contract::calculateSalary → *basic + 10% allowance (no tax)*

In main(), ask the user for the employee type and basic salary, call the matching namespace's function, and print a neatly formatted salary slip using setw, setfill, fixed, and setprecision(2) so amounts line up in two decimal places.

2. Write a program that ask the user how many students are in a class, use new to dynamically allocate an integer array of that size, and read in marks. Compute and display the highest, lowest, average, and number of students who passed (≥ 40). Then ask if the teacher wants to add more students: dynamically allocate a *new*, larger array, copy the old data into it, delete[] the old array, and continue. Finally delete[] the last array before exiting.

3. Write a program with overloaded area() functions (using inline for one and a default argument for another) that calculates the area of a square, rectangle, or triangle based on the number of arguments passed.

```
[ inline double area(double side) → square (1 parameter)
  double area(double length, double width) → rectangle (2 parameters)
  double area(double base, double height, double factor = 0.5) → triangle (3 parameters; factor
  defaults to 0.5)
]
```

4. Write a program that swaps two integers using a pointer-based function and a reference-based function, and shows why each does or doesn't change the original values in main().

```
[ swapByValue(int a, int b): ordinary value parameters
  swapByPointer(int *a, int *b): pointers are passed by value, but dereferencing them
                                swaps the actual caller variables
  swapByReference(int &a, int &b): reference parameters ]
```

5. Write a program that passes an array to a function to find its sum, maximum, and minimum, and another function to check whether the array is sorted.